For 8-puzzle A* implementation, to calculate, f(n), consider two cases,
1. g(n): Depth of the node, h(n): Number of Misplaced tiles

```
#For 8-puzzle A* implementation, to calculate, f(n), consider two cases,
# 1. g(n): Depth of the node, h(n): Number of Misplaced tiles

import heapq

goal = [1, 2, 3, 8, 0, 4, 7, 6, 5]

def heuristic(state):
    return sum(1 for i in range(9)
               if state[i] != 0 and state[i] != goal[i])

def get_neighbors(state):
    neighbors = []
    zero = state.index(0)
    row, col = divmod(zero, 3)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        r, c = row + dr, col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            new_pos = r * 3 + c
            s = state[:]
            s[zero], s[new_pos] = s[new_pos], s[zero]
            neighbors.append(s)

    return neighbors

def astar(start):
    pq = [(heuristic(start), 0, start)]
    visited = set()

    while pq:
        f, g, state = heapq.heappop(pq)

        if state == goal:
            print("Goal reached!")
            return

        key = tuple(state)
        if key in visited:
            continue
        visited.add(key)
```

```
        print("State:", state, "g =", g,
              "h =", heuristic(state), "f =", f)

        for next_state in get_neighbors(state):
            if tuple(next_state) not in visited:
                new_g = g + 1
                new_f = new_g + heuristic(next_state)
                heapq.heappush(
                    pq, (new_f, new_g, next_state)
                )

start = [2, 8, 3,
         1, 6, 4,
         7, 0, 5]

astar(start)
```

```
rs/BMSCE/Desktop/1BM24CS327/misplaced.py
State: [2, 8, 3, 1, 6, 4, 7, 0, 5] g = 0 h = 4 f = 4
State: [2, 8, 3, 1, 0, 4, 7, 6, 5] g = 1 h = 3 f = 4
State: [2, 0, 3, 1, 8, 4, 7, 6, 5] g = 2 h = 3 f = 5
State: [2, 8, 3, 0, 1, 4, 7, 6, 5] g = 2 h = 3 f = 5
State: [0, 2, 3, 1, 8, 4, 7, 6, 5] g = 3 h = 2 f = 5
State: [1, 2, 3, 0, 8, 4, 7, 6, 5] g = 4 h = 1 f = 5
Goal reached!
```